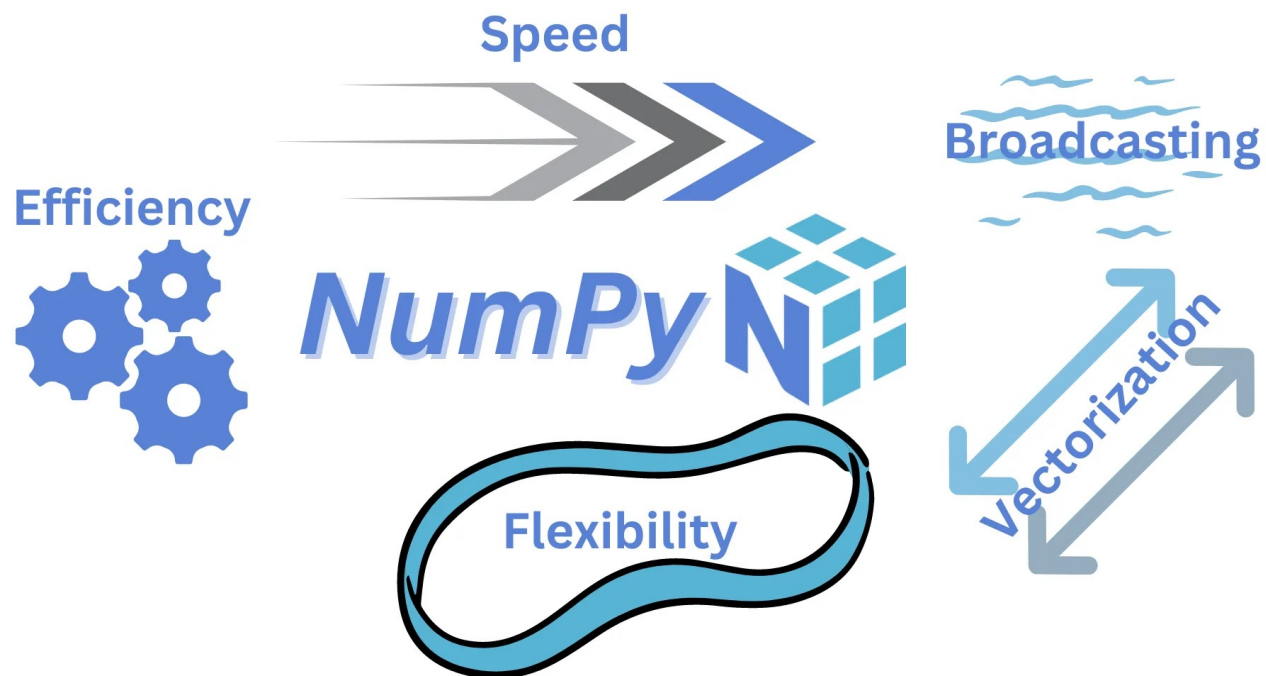


AI VIET NAM
@aivietnam.edu.vn

List and NumPy Array

(Performance Comparison)



Vinh Dinh Nguyen
PhD in Computer Science

Outline



➤ **List and NumPy Array**

➤ **2D List and 2D NumPy Array**

➤ **Kahoot Quizz**

➤ **IOU Calculation: Review**

➤ **IOU Calculate on N-Boxes: List and Array**

➤ **Non-Maxima Suppression: List and Array**

➤ **Summary**

List and NumPy Array

❖ List

```
list_name = [element-1, ..., element-n]
```

```
// create a list  
data = [6, 5, 7, 1, 9, 2]
```

data =	6	5	7	1	9	2
index	0	1	2	3	4	5

```
# Using list comprehension  
n = 5  
list_zeros = [0] * n  
print("List initialized with zeros:", list_zeros)
```

❖ NumPy Array

```
array_name = np.array([element-1, ..., element-n])
```

```
import numpy as np  
  
data = np.array([6, 5, 7, 1, 9, 2])
```

```
# Create a 1D NumPy array filled with zeros  
n = 5  
zeros_array_1d = np.zeros(n)  
print("1D NumPy array initialized with zeros:", zeros_array_1d)
```

List and NumPy Array

❖ Index

❖ List

```
l_data = [4, 5, 6, 7, 8, 9]
```

Forward
index

0	1	2	3	4	5
4	5	6	7	8	9

Backward
index

-6	-5	-4	-3	-2	-1
----	----	----	----	----	----

`l_data[0]`

4

`l_data[3]`

7

`l_data[-1]`

9

`l_data[-3]`

7

❖ NumPy Array

```
a_data = np.array([4, 5, 6, 7, 8, 9])
```

Forward
index

0	1	2	3	4	5
4	5	6	7	8	9

Backward
index

-6	-5	-4	-3	-2	-1
----	----	----	----	----	----

`a_data[0]`

4

`a_data[3]`

7

`a_data[-1]`

9

`a_data[-3]`

7

List and NumPy Array

❖ List

`list[start:end:step]`

`data = [4, 5, 6, 7, 8, 9]`



Forward
index



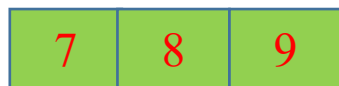
`data[:3]`



`data[2:4]`



`data[3:]`



5

Giá trị mặc định của start là 0, của end là len(list), và của step là 1

❖ Slicing

❖ NumPy Array

`array[start:end:step]`

`a_data = np.array([4, 5, 6, 7, 8, 9])`



Forward
index



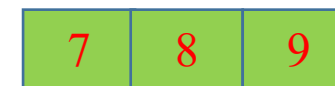
`a_data[:3]`



`a_data[2:4]`



`a_data[3:]`



5

Giá trị mặc định của start là 0, của end là len(list), và của step là 1

List and NumPy Array

❖ List

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

`data.append(4)` # thêm 4 vào vị trí cuối list

`data =`

6	5	7	1	9	2	4
---	---	---	---	---	---	---

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

`data.insert(0, 4)` # thêm 4 vào vị trí có
index = 0

`data =`

4	6	5	7	1	9	2
---	---	---	---	---	---	---

❖ Add an element

❖ NumPy Array

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

`a_new = np.append(a_data, 4)` # thêm 4 vào vị trí cuối array

`a_new =`

6	5	7	1	9	2	4
---	---	---	---	---	---	---

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

`a_new = np.insert(a_data, 0, 4)` # thêm 4 vào vị trí có
index = 0

`a_new =`

4	6	5	7	1	9	2
---	---	---	---	---	---	---

List and NumPy Array

❖ List

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

thay đổi phần tử thứ 1
`data[1] = 4`

`data =`

6	4	7	1	9	2
---	---	---	---	---	---

❖ Add a list of elements

`data =`

6	5	7	1
---	---	---	---

`data.extend([9, 2])` # thêm 9 và 2 vào vị trí cuối list

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

❖ Updating an element

❖ NumPy Array

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

thay đổi phần tử thứ 1
`a_data[1] = 4`

`a_data =`

6	4	7	1	9	2
---	---	---	---	---	---

❖ Add an array of elements

`a_data =`

6	5	7	1
---	---	---	---

`a_data = np.append(a_data, [9, 2])`
thêm 9 và 2 vào vị trí cuối list

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

List and NumPy Array

❖ List

data1 =

6	5	7
---	---	---

data2 =

1	9	2
---	---	---

nối 2 list

data = data1 + data2

data =

6	5	7	1	9	2
---	---	---	---	---	---

data =

6	5
---	---

nhân list với một số nguyên

data_m = data * 3

data_m =

6	5	6	5	6	5
---	---	---	---	---	---

❖ + and * operators

❖ NumPy Array

a_data1 =

6	5	7
---	---	---

a_data2 =

1	9	2
---	---	---

nối 2 list

a_data = np.append(a_data1, a_data2)

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

a_data =

6	5
---	---

nhân array với một số nguyên

data_m = data * 3

data_m =

18	5
----	---

List and NumPy Array

❖ List

❖ sort() – Sắp xếp các phần tử

data =

6	5	7	1	9	2
---	---	---	---	---	---

data.sort()

data =

1	2	5	6	7	9
---	---	---	---	---	---

data =

6	5	7	1	9	2
---	---	---	---	---	---

data.sort(reverse = True)

data =

9	7	6	5	2	1
---	---	---	---	---	---

❖ NumPy Array

❖ sort() – Sắp xếp các phần tử

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

a_data.sort()

a_data =

1	2	5	6	7	9
---	---	---	---	---	---

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

a_data.sort()

a_data = a_data[::-1]

data =

9	7	6	5	2	1
---	---	---	---	---	---

List and NumPy Array

❖ List

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

`data.pop(2)` # tại vị trí index = 2

`data =`

6	5	1	9	2
---	---	---	---	---

`data =`

6	5	7	1	9	2
---	---	---	---	---	---

`data.remove(5)` # xóa phần tử đầu tiên
có giá trị là 5

`data =`

6	7	1	9	2
---	---	---	---	---

❖ Deleting an element

❖ NumPy Array

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

`a_data = np.delete(a_data, 2)` # tại vị trí index = 2

`a_data =`

6	5	1	9	2
---	---	---	---	---

`a_data =`

6	5	7	1	9	2
---	---	---	---	---	---

`deleted_index = np.where(data == 5)[0][0]`
`a_data = np.delete(a_data, deleted_index)`
xóa phần tử đầu tiên
có giá trị là 5

`a_data =`

6	7	1	9	2
---	---	---	---	---

List and NumPy Array

❖ List

```
data = [6, 5, 7, 1, 9, 2]
```

xóa phần tử thứ 1 và 2

```
del data[1:3]
```

```
data = [6, 1, 9, 2]
```

❖ Delete elements

❖ NumPy Array

```
a_data = [6, 5, 7, 1, 9, 2]
```

xóa phần tử thứ 1 và 2

```
start_index = 1
```

```
end_index = 3
```

```
a_data = np.concatenate(  
    (a_data[:start_index],  
     a_data[end_index:]))
```

```
a_data = [6, 1, 9, 2]
```

```
a_data = np.delete(a_data, np.arange(1,3))
```

List and NumPy Array

❖ List

index() – Trả về vị trí đầu tiên

data =

6	5	7	1	9	2
---	---	---	---	---	---

trả về vị trí của phần tử đầu tiên có giá trị là 9

data.index(9) = 4

❖ NumPy Array

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

element_to_find = 9

index = **np.where(arr == element_to_find)[0][0]**

List and NumPy Array

❖ List

count() – Trả về số lần xuất hiện của một phần tử

data =

6	5	7	1	9	2
---	---	---	---	---	---

trả về số lần phần tử 7 xuất hiện trong list
data.count(7) = 1

copy() – copy một list

data =

6	5	7	1	9	2
---	---	---	---	---	---

data_copy = data.copy()

data_copy =

6	5	7	1	9	2
---	---	---	---	---	---

❖ NumPy Array

count() – Trả về số lần xuất hiện của một phần tử

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

trả về số lần phần tử 7 xuất hiện trong list
count = (a_data == 3).sum()

copy() – copy một array

data =

6	5	7	1	9	2
---	---	---	---	---	---

data_copy = data.copy()

data_copy =

6	5	7	1	9	2
---	---	---	---	---	---

List and NumPy Array

❖ Built in Functions

❖ List

len(), min(), and max()

data =

6	5	7	1	9	2
---	---	---	---	---	---

trả về số phần tử

len(data) = 6

trả về số phần tử có giá trị nhỏ nhất

min(data) = 1

trả về số phần tử có giá trị lớn nhất

max(data) = 9

❖ NumPy Array

len(), min(), and max()

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

trả về số phần tử

len(a_data) = 6

trả về số phần tử có giá trị nhỏ nhất

np.min(a_data) = 1

trả về số phần tử có giá trị lớn nhất

np.max(a_data) = 9

trả về số chiều của array

a_data.shape = (6,)



List and NumPy Array

❖ Built in Functions

sum()

❖ List

$$summation = \sum_{i=0}^n data_i$$

data =

6	5	7	1	9	2
---	---	---	---	---	---

tính tổng

sum(data) = 30

```
1 data = [6, 5, 7, 1, 9, 2]
2 print(data)
3
4 summation = sum(data)
5 print(summation)
```

[6, 5, 7, 1, 9, 2]
30

❖ NumPy Array

a_data =

6	5	7	1	9	2
---	---	---	---	---	---

tính tổng

np.sum(data) = 30

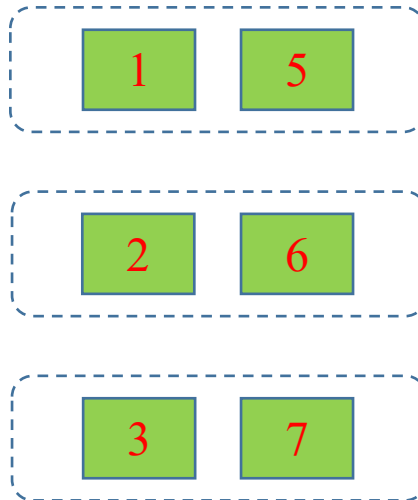
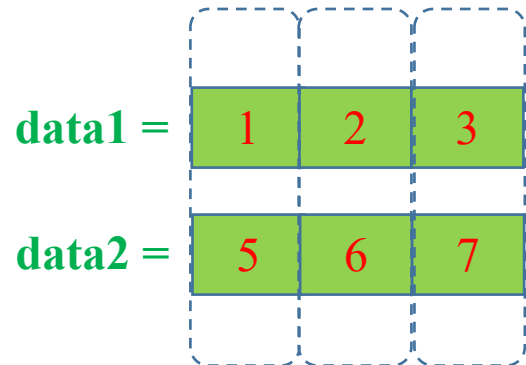
```
1 import numpy as np
2
3 data = np.array([6, 5, 7, 1, 9, 2])
4 print(data)
5
6 summation = np.sum(data)
7 print(summation)
```

[6, 5, 7, 1, 9, 2]
30

List

❖ Built in Functions for List

zip()



```

1 l1 = [1, 2, 3]
2 l2 = [5, 6, 7]
3
4 # print in pairs
5 length = len(l1)
6 for i in range(length):
7     print(l1[i], l2[i])

```

```

1 5
2 6
3 7

```

```

1 l1 = [1, 2, 3]
2 l2 = [5, 6, 7]
3
4 # print in pairs
5 for v1, v2 in zip(l1, l2):
6     print(v1, v2)

```

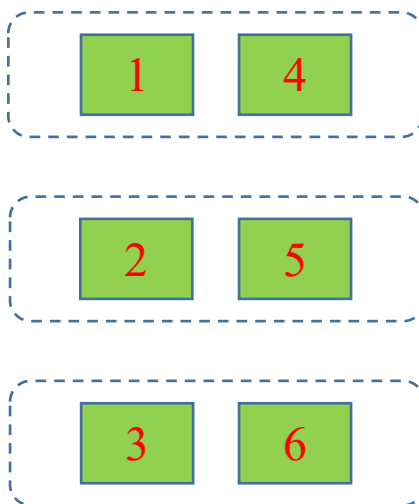
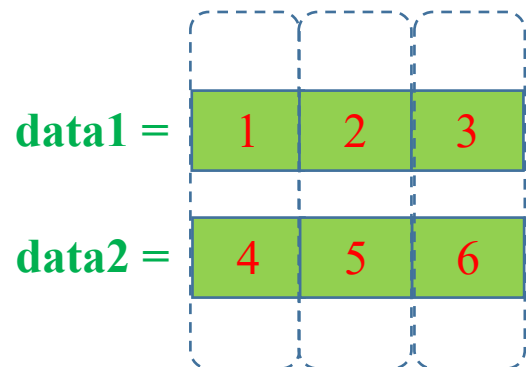
```

1 5
2 6
3 7

```


❖ Built in Functions

No zip() function



```
1 import numpy as np
2
3 # Example arrays
4 arr1 = np.array([1, 2, 3])
5 arr2 = np.array([4, 5, 6])
6
7 # Zip-like operation using numpy.dstack()
8 zipped_arr = np.dstack((arr1, arr2))[0]
9 print(len(zipped_arr))
10 for i in range(len(arr1)):
11     print(zipped_arr[i])
```

Output

```
3
[1 4]
[2 5]
[3 6]
```

❖ Built in Functions for List **enumerate()**

data =

6	1	7
---	---	---

enumerate(data) =

6	1	7
---	---	---

index 0 1 2

```
1  # get index and value
2  data = [6, 1, 7]
3
4  length = len(data)
5  for index in range(length):
6      print(index, data[index])
```

0 6
1 1
2 7

```
1  # enumerate
2  data = [6, 1, 7]
3  for index, value in enumerate(data):
4      print(index, value)
```

0 6
1 1
2 7

❖ Built in Functions

np.ndenumerate()

data =

6	1	7
---	---	---

ndenumerate(data) =

6	1	7
---	---	---

Tuple index (0,) (1,) (2,)

```
1 import numpy as np
2
3 # Example arrays
4 arr1 = np.array([6, 1, 7])
5
6 for index,value in np.ndenumerate(arr1):
7     print(index,value)
```

Output

(0,) 6
(1,) 1
(2,) 7

List Examples

Sum of even numbers

data =

6	5	7	1	9	2
---	---	---	---	---	---

```
1 # sum of even number
2 def sum1(data):
3     result = 0
4
5     for value in data:
6         if value%2 == 0:
7             result = result + value
8
9     return result
10
11 # test
12 data = [6, 5, 7, 1, 9, 2]
13 summation = sum1(data)
14 print(summation)
```

Sum of elements with even indices

data =

6	5	7	1	9	2
---	---	---	---	---	---

```
1 # sum of numbers with even indices
2 def sum2(data):
3     result = 0
4
5     length = len(data)
6     for index in range(length):
7         if index%2 == 0:
8             result = result + data[index]
9
10    return result
11
12 # test
13 data = [6, 5, 7, 1, 9, 2]
14 summation = sum2(data)
15 print(summation)
```



NumPy Array Examples

Sum of even numbers

```
1 import numpy as np
2
3 # Create a NumPy array
4 arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
5
6 # Use boolean indexing to find even numbers
7 even_numbers = arr[arr % 2 == 0]
8
9 # Calculate the sum of even numbers
10 sum_even_numbers = np.sum(even_numbers)
11
12 print("Original array:", arr)
13 print("Even numbers:", even_numbers)
14 print("Sum of even numbers:", sum_even_numbers)
```

Sum of elements with even indices

```
1 import numpy as np
2
3 # Create a NumPy array
4 arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
5
6 # Select elements at even indices
7 even_index_elements = arr[::2] # This selects every second element
   starting from index 0
8
9 # Calculate the sum of elements at even indices
10 sum_even_indices = np.sum(even_index_elements)
11
12 print("Original array:", arr)
13 print("Elements at even indices:", even_index_elements)
14 print("Sum of elements at even indices:", sum_even_indices)
```



Advanced Indexing

Integer Array Indexing: Construct new arrays from another array.

```
a = np.array([1, 2, 3, 4, 5])  
print(a[[1, 3]]) # Outputs: [2, 4]
```

Boolean Array Indexing: Conditionally select elements.

```
a = np.array([1, 2, 3, 4, 5])  
print(a[a > 3]) # Outputs: [4, 5]
```

Array Manipulation

```
# Reshaping: Change the shape of an array without changing its data.  
arr = np.array([[1, 2], [3, 4], [5, 6]])  
print(arr.reshape(2, 3))  
# [[1 2 3]  
#   [4 5 6]]  
  
# Flattening: Convert a multi-dimensional array into a 1D array.  
arr = np.array([[1, 2], [3, 4]])  
print(arr.flatten()) # [1 2 3 4]  
  
# Concatenation: Joining two or more arrays.  
a = np.array([1, 2])  
b = np.array([3, 4])  
print(np.concatenate((a, b))) # [1 2 3 4]
```

Broadcasting and Vectorization in NumPy

Broadcasting: The Unsung Hero

Making arrays with mismatched shapes compatible for arithmetic operations.

It “extends” smaller arrays to the shape of the larger array,

Element-wise operations on arrays of different shapes

```
import numpy as np
a = np.array([[1, 2], [3, 4], [5, 6]])
b = np.array([1, 2])

# Broadcasting in action: adding the 1D array 'b' to each row of the 2D array
result = a + b
# Outputs:
# [[2 4]
#  [4 6]
#  [6 8]]
```



Broadcasting and Vectorization in NumPy

Vectorization: Powering Efficient Computations

NumPy vectorization involves performing mathematical operations on entire arrays, eliminating the need to loop through individual elements.

The execution of operations on entire arrays, eliminating the need for explicit loops

Making calculations significantly faster than traditional Python loops.

Almost all NumPy operations are inherently vectorized.

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

# Vectorized multiplication of the two arrays
result = a * b
# Outputs: [5 12 21 32]
```


Outline



➤ **List and NumPy Array**

➤ **2D List and 2D NumPy Array**

➤ **Kahoot Quizz**

➤ **IOU Calculation: Review**

➤ **IOU Calculate on N-Boxes: List and Array**

➤ **Non-Maxima Suppression: List and Array**

➤ **Summary**

2D List & 2D Array

```
1 # Create a 2D list
2 list_2d = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 print(list_2d)
```

Output

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
1 import numpy as np
2
3 # Create a 2D NumPy array
4 array_2d = np.array([
5     [1, 2, 3],
6     [4, 5, 6],
7     [7, 8, 9]
8 ])
9
10 # Accessing elements in a 2D NumPy array
11 print("\n2D NumPy Array:")
12 print(array_2d)
```

Output

```
2D NumPy Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

2D List & 2D Array

```
1 # Create a 2D list
2 list_2d = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 print(list_2d)
```

Output

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Accessing elements in 2D list

```
print("Element at (1,2) in list_2d:", list_2d[1][2])
```

```
1 import numpy as np
2
3 # Create a 2D NumPy array
4 array_2d = np.array([
5     [1, 2, 3],
6     [4, 5, 6],
7     [7, 8, 9]
8 ])
9
10 # Accessing elements in a 2D NumPy array
11 print("\n2D NumPy Array:")
12 print(array_2d)
```

Output

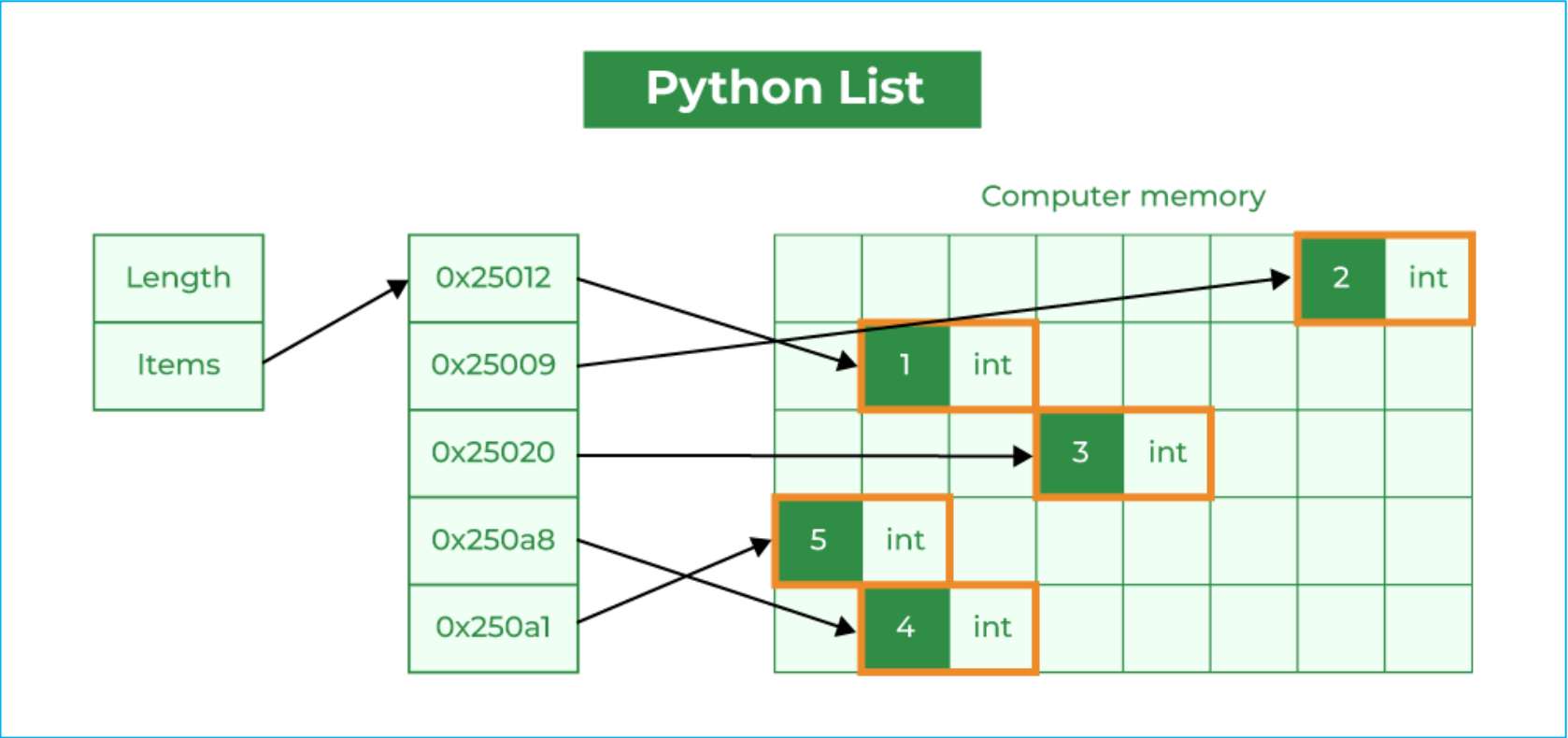
2D NumPy Array:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Accessing elements in 2D NumPy array

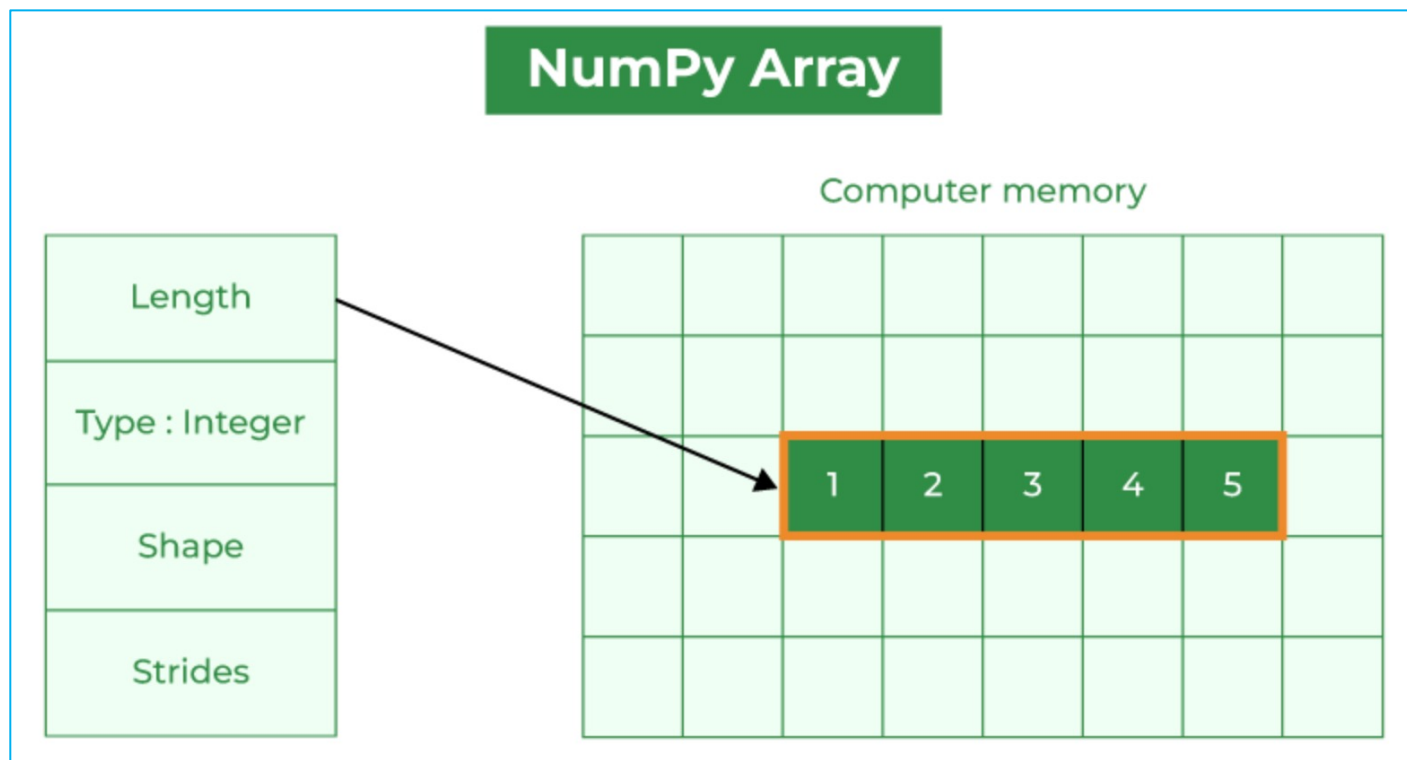
```
print("Element at (1,2) in array_2d:", array_2d[1, 2])
```

Python List Limitations



- 1.Element Overhead
- 2.Memory Fragmentation
- 3.Performance
- 4.Functionality

NumPy Array Motivation



1. Homogeneous Data
2. Fixed Data Type
3. Contiguous Memory
4. Performance

1D List/Array Multiplication

```
array_name = np.array[element-1, ..., element-n]
```

```
list_name = [element-1, ..., element-n]
```



```
def multiplizeList(size=1000):  
    # declaring lists  
    list1 = list(range(size))  
    list2 = list(range(size))  
  
    # capturing time before the multiplication of Python lists  
    initialTime = time.time()  
  
    # multiplying elements of both the lists and stored in another list  
    resultantList = [(a * b) for a, b in zip(list1, list2)]  
  
    # calculating execution time  
    print("Time taken by Lists to perform multiplication:",  
          (time.time() - initialTime),  
          "seconds")
```

```
def multiplizeArray(size=1000):  
    # declaring arrays  
    array1 = numpy.arange(size)  
    array2 = numpy.arange(size)  
  
    # capturing time before the multiplication of Numpy arrays  
    initialTime = time.time()  
  
    # multiplying elements of both the Numpy arrays and stored in another Numpy array  
    resultantArray = array1 * array2  
  
    # calculating execution time  
    print("Time taken by NumPy Arrays to perform multiplication:",  
          (time.time() - initialTime),  
          "seconds")
```

```
multiplizeList(100000)  
multiplizeArray(100000)
```

```
Time taken by Lists to perform multiplication: 0.025510549545288086 seconds  
Time taken by NumPy Arrays to perform multiplication: 0.0005364418029785156 seconds
```

2D List/Array Multiplication

❖ Python List Solution

```
def matrix_multiply(A, B):
    # Get the number of rows and columns for the input matrices
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])

    # Ensure the number of columns in A is equal to the number of rows in B
    if cols_A != rows_B:
        raise ValueError("Number of columns in A must be equal to number of rows in B")

    # Initialize the result matrix with zeros
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Perform the matrix multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix} = \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix} = \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

```
# Example matrices
A = [[1, 2, 3],
     [4, 5, 6]]

B = [[7, 8],
     [9, 10],
     [11, 12]]

# Multiply the matrices
result = matrix_multiply(A, B)

# Print the result
for row in result:
    print(row)
```

⇒ [58, 64]
[139, 154]

2D List/Array Multiplication

❖ NumPy Array Solution

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$
$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$
$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

```
import numpy as np

# Define the matrices
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

B = np.array([
    [7, 8],
    [9, 10],
    [11, 12]
])

# Multiply the matrices using np.dot
result = np.dot(A, B)

# Alternatively, you can use the @ operator
# result = A @ B

# Print the result
print(result)
```


Shape of NumPy Arrays

Python

```
>>> import numpy as np
>>> numbers = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
>>> numbers
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
```

Python

```
>>> numbers.shape
(2, 4)
```

Python

```
>>> numbers.ndim
2
```

1	2	3	4
5	6	7	8

shape
(2, 4)

Reshape in NumPy Arrays

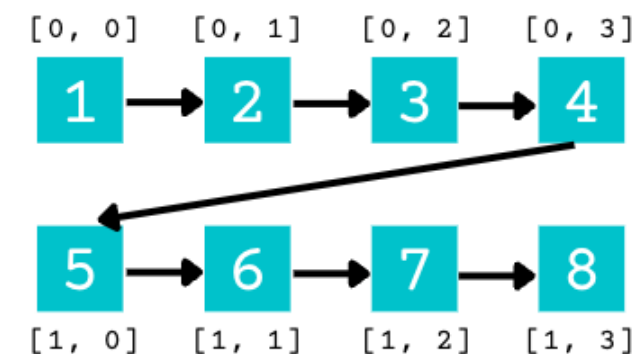
Python

```
>>> import numpy as np
>>> numbers = np.array([1, 2, 3, 4, 5, 6, 7, 8])
>>> numbers
array([1, 2, 3, 4, 5, 6, 7, 8])
>>> numbers.shape
(8,)
>>> numbers.ndim
1

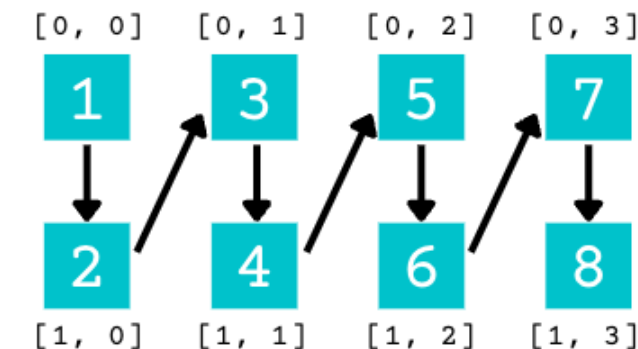
>>> numbers.reshape((2, 4), order="C")
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])

>>> numbers.reshape((2, 4), order="F")
array([[1, 3, 5, 7],
       [2, 4, 6, 8]])
```

C-like index order



Fortran-like index order



Outline



➤ **List and NumPy Array**

➤ **2D List and 2D NumPy Array**

➤ **Kahoot Quizz**

➤ **IOU Calculation: Review**

➤ **IOU Calculate on N-Boxes: List and Array**

➤ **Non-Maxima Suppression: List and Array**

➤ **Summary**

Outline

➤ **List and NumPy Array**

➤ **2D List and 2D NumPy Array**

➤ **Kahoot Quizz**

➤ **IOU Calculation: Review**

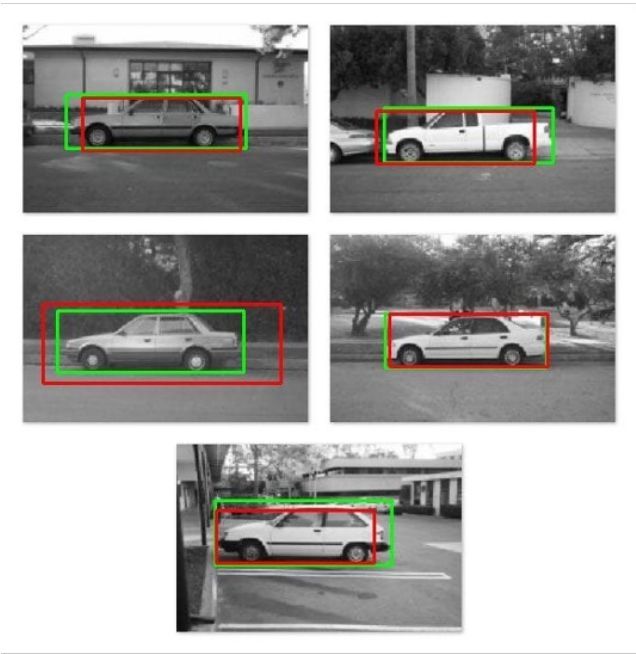
➤ **IOU Calculate on N-Boxes: List and Array**

➤ **Non-Maxima Suppression: List and Array**

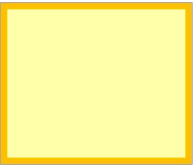
➤ **Summary**



IOU Calculation Using Tuple



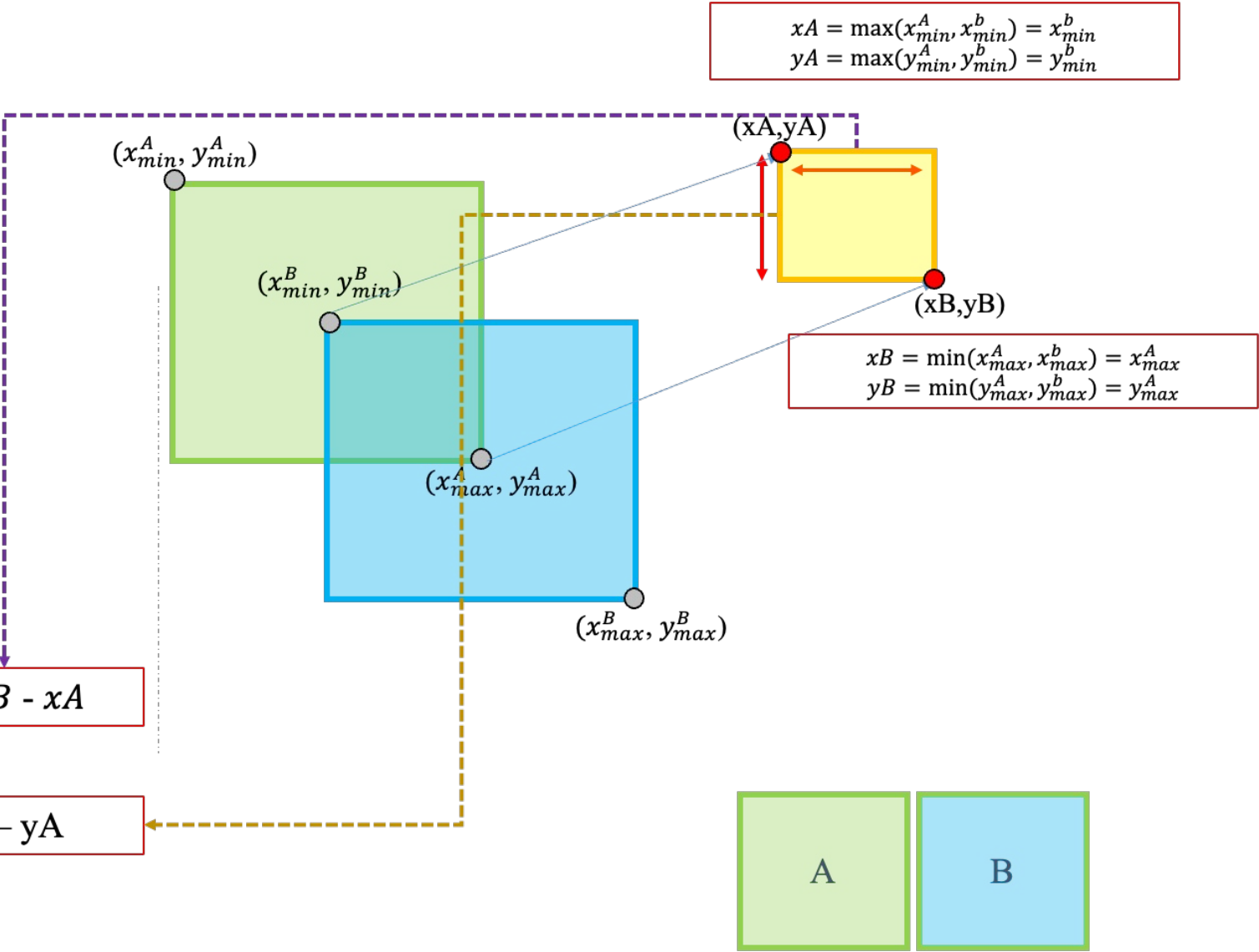
$A \cap B = W * H$



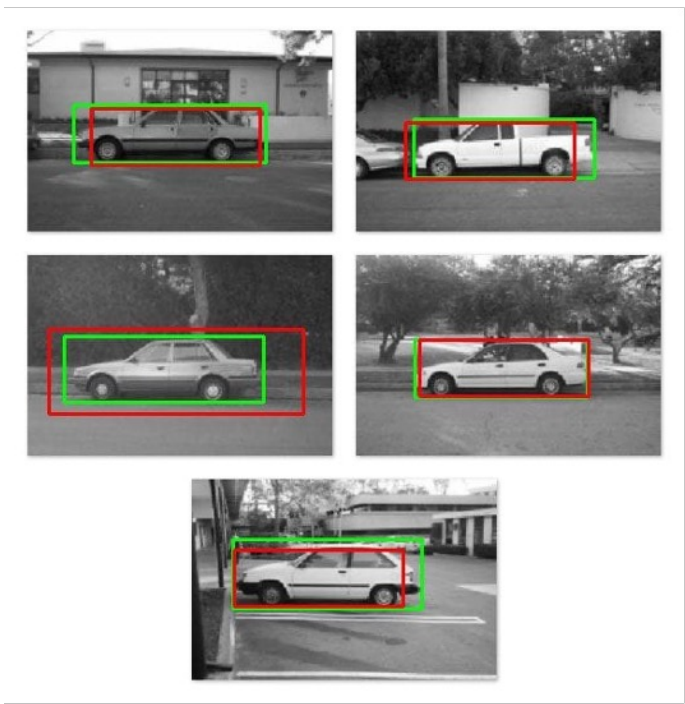
Area

$W = xB - xA$

$H = yB - yA$



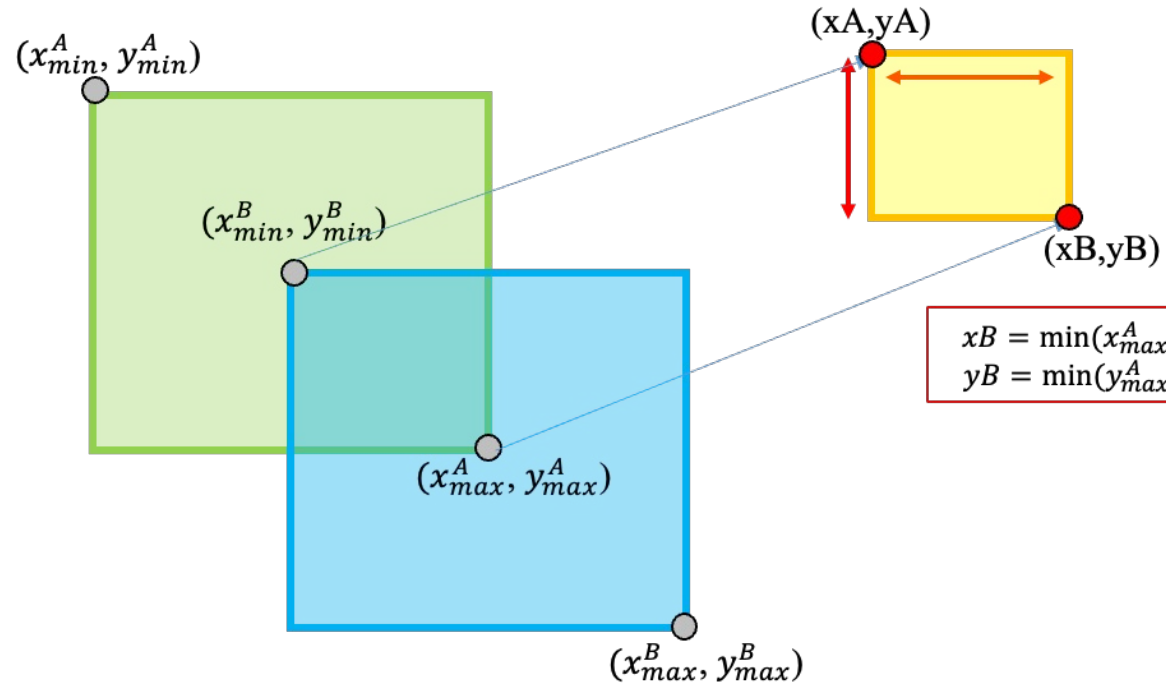
IOU Calculation Using Tuple



Giả sử ta có 2 boxes với thông tin như sau:

- Box A có tọa độ $(x_{min}^A, y_{min}^A, x_{max}^A, y_{max}^A) \Rightarrow \text{Tuple}$
- Box B có tọa độ $(x_{min}^B, y_{min}^B, x_{max}^B, y_{max}^B) \Rightarrow \text{Tuple}$

$$xA = \max(x_{min}^A, x_{min}^B) = x_{min}^B$$
$$yA = \max(y_{min}^A, y_{min}^B) = y_{min}^B$$



$$xB = \min(x_{max}^A, x_{max}^B) = x_{max}^A$$
$$yB = \min(y_{max}^A, y_{max}^B) = y_{max}^A$$

$(x_{max}^B - x_{min}^B) * (y_{max}^B - y_{min}^B)$

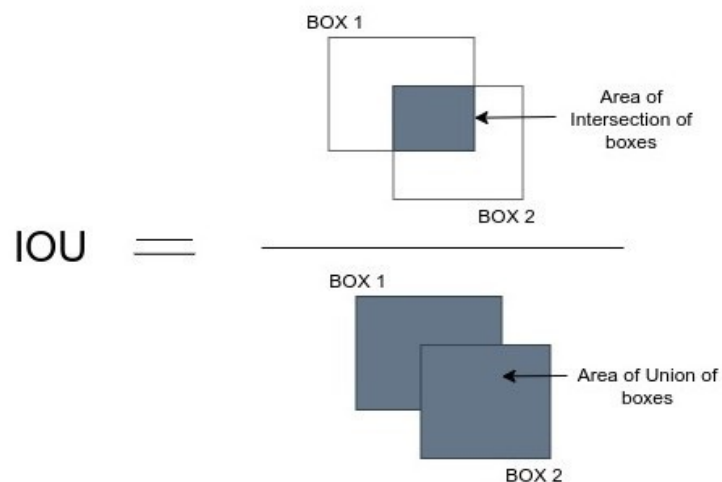
Area Area Area

$A \cup B =$ A $+$ B $-$

$(x_{max}^A - x_{min}^A) * (y_{max}^A - y_{min}^A)$



IOU Calculation Using Tuple



$$\frac{|A \cap B|}{|A \cup B|}$$

Tính diện tích của 2 box A, B

$$\text{area1} = (x_{\max}^A - x_{\min}^A) * (y_{\max}^A - y_{\min}^A)$$

$$\text{area2} = (x_{\max}^B - x_{\min}^B) * (y_{\max}^B - y_{\min}^B)$$

Tìm tọa độ của vùng giao nhau (Intersection)

$$xA = \max(x_{\min}^A, x_{\min}^B) = x_{\min}^B$$

$$yA = \max(y_{\min}^A, y_{\min}^B) = y_{\min}^B$$

$$xB = \min(x_{\max}^A, x_{\max}^B) = x_{\max}^A$$

$$yB = \min(y_{\max}^A, y_{\max}^B) = y_{\max}^A$$

Tính diện tích vùng giao nhau

$$W = xB - xA$$

$$H = yB - yA$$

$$\text{intersection_area} = w * h$$

Tính diện tích phần hợp nhau

$$\text{union_area} = \text{area1} + \text{area2} - \text{intersection_area}$$

Dựa trên phần giao và phần hợp để tính IoU

$$\text{IoU} = \text{intersection_area} / \text{union_area}$$

IOU Implementation

```

def computeIoU(boxA, boxB):
    # determine the (x, y)-coordinates of the intersection rectangle
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])

    # compute the area of intersection rectangle
    interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
    # compute the area of both the prediction and ground-truth
    # rectangles
    boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
    boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
    # compute the intersection over union by taking the intersection
    # area and dividing it by the sum of prediction + ground-truth
    # areas - the interesection area
    iou = interArea / float(boxAArea + boxBArea - interArea)
    # return the intersection over union value
    return iou

```

Tính diện tích của 2 box A, B

$$\text{area1} = (x_{\max}^A - x_{\min}^A) * (y_{\max}^A - y_{\min}^A)$$

$$\text{area2} = (x_{\max}^B - x_{\min}^B) * (y_{\max}^B - y_{\min}^B)$$

Tìm tọa độ của vùng giao nhau (Intersection)

$$x_A = \max(x_{\min}^A, x_{\min}^B) = x_{\min}^B$$

$$y_A = \max(y_{\min}^A, y_{\min}^B) = y_{\min}^B$$

$$x_B = \min(x_{\max}^A, x_{\max}^B) = x_{\max}^A$$

$$y_B = \min(y_{\max}^A, y_{\max}^B) = y_{\max}^A$$

Tính diện tích vùng giao nhau

$$W = x_B - x_A$$

$$H = y_B - y_A$$

$$\text{intersection_area} = w * h$$

Tính diện tích phần hợp nhau

$$\text{union_area} = \text{area1} + \text{area2} - \text{intersection_area}$$

Dựa trên phần giao và phần hợp để tính IoU

$$\text{IoU} = \text{intersection_area} / \text{union_area}$$

Outline

- **List and NumPy Array**
- **2D List and 2D NumPy Array**
- **Kahoot Quizz**
- **IOU Calculation: Review**
- **IOU Calculate on N-Boxes: List and Array**
- **Non-Maxima Suppression: List and Array**
- **Summary**



IOU Calculation Of N-Boxes: List

Box A → [12, 84, 140, 212],
Box B → [24, 84, 152, 212],
Box C → [36, 84, 164, 212],
Box D → [12, 96, 140, 224],
Box E → [24, 96, 152, 224],
]

```
# Example usage
boxes = [

```

```
# Example usage
boxes = []
for i in range(10000):
    boxes.append([i+1, i+1, i+1, i+1])

initialTime = time.time()

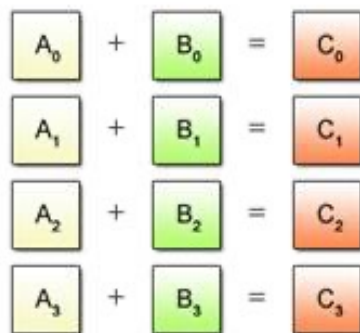
for i in range(1, len(boxes)):
    iou = computeIoU(boxes[0], boxes[i])
    # print("iou", iou)

# calculating execution time
print("Time taken by NumPy Arrays to perform multiplication:",
      (time.time() - initialTime),
      "seconds")
```

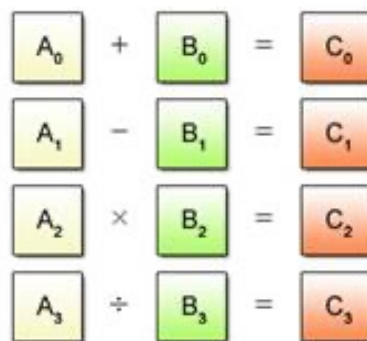
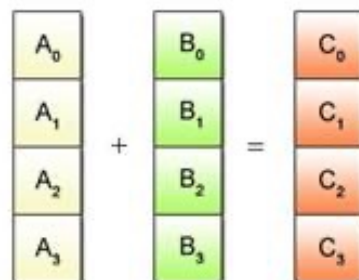
⇒ Time taken by NumPy Arrays to perform multiplication: 0.023508548736572266 seconds

NumPy Array

(a) Scalar Operation



(b) SIMD Operation



Example of SIMD Unprocesable Patterns

```
print("Time taken by NumPy Arrays to perform multiplication:",
      (time.time() - initialTime),
      "seconds")
```

Time taken by NumPy Arrays to perform multiplication: 0.0019757747650146484 seconds

```
# Array solution
```

```
initialTime = time.time()
```

```
boxes = np.array(boxes)
```

```
scores = np.array(scores)
```

```
# Coordinates of bounding boxes
```

```
x1 = boxes[:, 0]
```

```
y1 = boxes[:, 1]
```

```
x2 = boxes[:, 2]
```

```
y2 = boxes[:, 3]
```

```
# Compute the area of the bounding boxes
```

```
areas = (x2 - x1 + 1) * (y2 - y1 + 1)
```

```
# Compute the intersection areas
```

```
xx1 = np.maximum(x1[0], x1[1:])
```

```
yy1 = np.maximum(y1[0], y1[1:])
```

```
xx2 = np.minimum(x2[0], x2[1:])
```

```
yy2 = np.minimum(y2[0], y2[1:])
```

```
w = np.maximum(0, xx2 - xx1 + 1)
```

```
h = np.maximum(0, yy2 - yy1 + 1)
```

```
inter = w * h
```

```
# # Compute the IoU
```

```
iou = inter / (areas[0] + areas[1:] - inter)
```

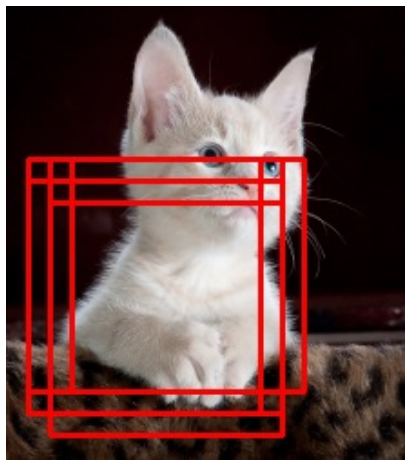
```
print("Time taken by NumPy Arrays to perform multiplication:",
      (time.time() - initialTime),
      "seconds")
```

Outline

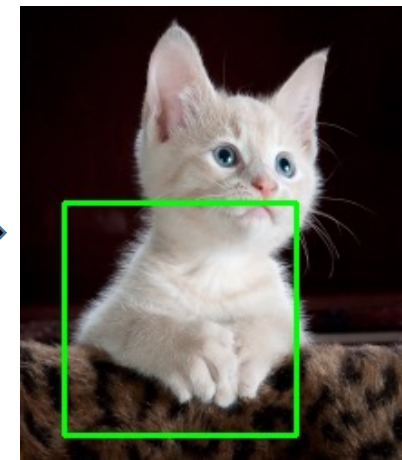
- **List and NumPy Array**
- **2D List and 2D NumPy Array**
- **Kahoot Quizz**
- **IOU Calculation: Review**
- **IOU Calculate on N-Boxes: List and Array**
- **Non-Maxima Suppression: List and Array**
- **Summary**



Non-Maxima Suppression



Non Maximum Suppression



List

Tuple

Algorithm

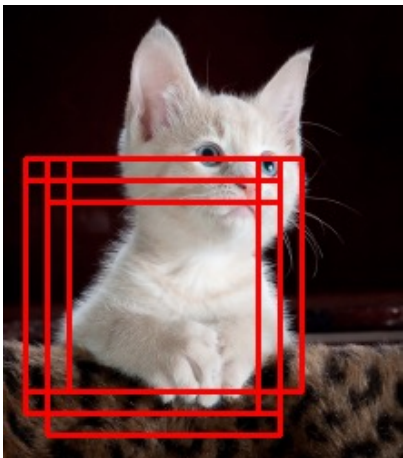
```
# Example usage
boxes = [
    [12, 84, 140, 212],
    [24, 84, 152, 212],
    [36, 84, 164, 212],
    [12, 96, 140, 224],
    [24, 96, 152, 224],
    [24, 108, 152, 236]
]
```

Step 1 : Select the prediction **S** with highest confidence score and remove it from **P** and add it to the final prediction list **keep**. (**keep** is empty initially).

Step 2 : Now compare this prediction **S** with all the predictions present in **P**. Calculate the IoU of this prediction **S** with every other predictions in **P**. If the IoU is greater than the threshold **thresh_iou** for any prediction **T** present in **P**, remove prediction **T** from **P**.

Step 3 : If there are still predictions left in **P**, then go to **Step 1** again, else return the list **keep** containing the filtered predictions.

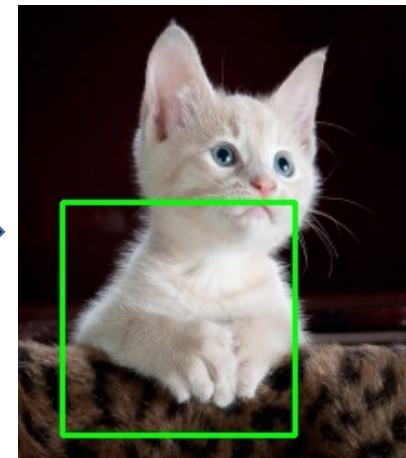
Non-Maxima Suppression



```
# Example usage
boxes = [
    [12, 84, 140, 212],
    [24, 84, 152, 212],
    [36, 84, 164, 212],
    [12, 96, 140, 224],
    [24, 96, 152, 224],
]
```



Non Maximum Suppression



```
# Example usage
boxes = [
    [24, 108, 152, 236]
]
```

```
boxes = [
    [12, 84, 140, 212],
    [24, 84, 152, 212],
    [36, 84, 164, 212],
    [12, 96, 140, 224],
    [24, 96, 152, 224],
    [24, 108, 152, 236]
]

scores = [0.3, 0.4, 0.5, 0.6, 0.7]
iou_threshold = 0.3

kept_indices = non_max_suppression(boxes, scores, iou_threshold)
kept_boxes = [boxes[i] for i in kept_indices]

print("Kept indices:", kept_indices)
print("Kept boxes:", kept_boxes)
```

List Implementation



NumPy Array Implementation



Outline

- **List and NumPy Array**
- **2D List and 2D NumPy Array**
- **Kahoot Quizz**
- **IOU Calculation: Review**
- **IOU Calculate on N-Boxes: List and Array**
- **Non-Maxima Suppression: List and Array**
- **Summary**





Summary

Feature	Python List	NumPy Array
Storage	Dynamic type	Fixed type
Functionality	General-purpose	Mathematical and scientific operations
Speed	Slower due to type-checking and overhead	Faster due to vectorized operations and no type-checking
Memory	More due to overhead	Optimized and efficient storage

Python List Operation	NumPy Array Operation	Description
<code>`len(list)`</code>	<code>`array.shape`</code>	Get number of elements
<code>`list.append(item)`</code>	N/A	Add an item (NumPy arrays are fixed-size)
<code>`for item in list`</code>	Vectorized operations	Apply operation to each item
<code>`list1 + list2`</code>	<code>`array1 + array2`</code>	Concatenate lists vs. element-wise addition
<code>`sum(list)`</code>	<code>`np.sum(array)`</code>	Sum of all elements

References

Problem Solving with Algorithms and Data Structures

Release 3.0

Brad Miller, David Ranum

September 22, 2013

Python Data Structures and Algorithms

Improve the performance and speed of your applications



Packt>

